

Enterprise Management System

SQL Queries

Q1

```
SELECT department_id, COUNT(*) AS employee_count, AVG(salary) AS average_salary
FROM employees GROUP BY department_id;
```

Output

department_id	employee_count	average_salary
NULL	1	50000
1	4	84250
2	2	55000
3	3	78333.33

Q2

```
SELECT department_id, AVG(salary) AS average_salary
FROM employees GROUP BY department_id HAVING (AVG(salary)>70000);
```

Output

department_id	average_salary
1	84250
3	78333.33

Q3

```
SELECT e.employee_name, d.department_name, e.salary
FROM employees e
INNER JOIN departments d
ON e.department_id = d.department_id;
```

Output

employee_name	department_name	salary
Amit	Engineering	85000
Rahul	Engineering	72000
Priya	HR	65000
Neha	Finance	95000
Amit	Engineering	88000
Anita	Engineering	92000
Vijay	HR	45000
Kiran	Finance	78000
Pooja	Finance	62000

Q4

```
SELECT e.employee_name
FROM employees e
LEFT JOIN departments d
ON e.department_id = d.department_id
WHERE d.department_id IS NULL;
```

Output

employee_name
Ravi

Q5

```
SELECT employee_id
FROM projects
GROUP BY employee_id
HAVING COUNT(*) > 2;
```

Output

employee_id
101

Q6

```
SELECT employee_id,
       SUM(project_cost) AS total_cost
FROM projects
GROUP BY employee_id;
```

Output

employee_id	total_cost
101	1250000
102	300000
103	1200000
104	50000

Q7

```
SELECT *
FROM employees
ORDER BY salary DESC
LIMIT 5;
```

Output

employee_id	employee_name	department_id	salary	joining_date	city
104	Neha	3	95000	2024-11-05	Delhi
107	Anita	1	92000	2023-02-14	Bangalore
105	Amit	1	88000	2026-03-15	Mumbai
101	Amit	1	85000	2026-03-15	Delhi
109	Kiran	3	78000	2025-09-20	Bangalore

Q8

```
SELECT MAX(salary)
FROM employees
WHERE salary < (
    SELECT MAX(salary)
    FROM employees
);
```

Output

MAX(salary)
92000

Q9

```
SELECT *  
FROM employees  
WHERE salary > (  
    SELECT AVG(salary)  
    FROM employees  
);
```

Output

employee_id	employee_name	department_id	salary	joining_date	city
101	Amit	1	85000	2026-03-15	Delhi
104	Neha	3	95000	2024-11-05	Delhi
105	Amit	1	88000	2026-03-15	Mumbai
107	Anita	1	92000	2023-02-14	Bangalore
109	Kiran	3	78000	2025-09-20	Bangalore

Q10

```
SELECT department_id  
FROM employees  
GROUP BY department_id  
HAVING COUNT(*) >= 5;
```

Output

(Empty result — 0 rows)

Q11

```
SELECT employee_name  
FROM employees  
GROUP BY employee_name  
HAVING COUNT(*) > 1;
```

Output

employee_name
Amit

Q12

```
SELECT city  
FROM employees  
GROUP BY city  
HAVING COUNT(*) > 10;
```

Output

(Empty result — 0 rows)

Q13

```
SELECT employee_id
FROM projects
GROUP BY employee_id
HAVING SUM(project_cost) > 1000000;
```

Output

employee_id
101
103

Q14

```
SELECT e.employee_id,
       e.employee_name,
       SUM(p.project_cost) AS total_cost
FROM employees e
JOIN projects p
ON e.employee_id = p.employee_id
GROUP BY e.employee_id, e.employee_name;
```

Output

employee_id	employee_name	total_cost
101	Amit	1250000
102	Rahul	300000
103	Priya	1200000
104	Neha	50000

Q15

```
SELECT customer_id
FROM transactions
GROUP BY customer_id
HAVING SUM(amount) > 200000;
```

Output

customer_id
1
2
3

Q16

```
SELECT YEAR(transaction_date),
       MONTH(transaction_date),
       SUM(amount)
FROM transactions
GROUP BY YEAR(transaction_date),
         MONTH(transaction_date);
```

Output

year	month	SUM(amount)
2026	1	395000
2026	2	310000
2026	3	30000

Q17

```
SELECT customer_id,  
       SUM(amount)  
FROM transactions  
WHERE transaction_type = 'CREDIT'  
GROUP BY customer_id;
```

Output

customer_id	SUM(amount)
1	210000
2	250000
3	225000
4	30000

Q18

```
SELECT customer_id  
FROM transactions  
GROUP BY customer_id  
HAVING COUNT(*) > 5;
```

Output

(Empty result — 0 rows)

Q19

```
SELECT c.customer_name, t.amount  
FROM customers c  
JOIN transactions t  
ON c.customer_id = t.customer_id;
```

Output

customer_name	amount
Amit	50000
Amit	20000
Amit	160000
Rahul	250000
Priya	75000
Priya	150000
Neha	30000

Q20

```
SELECT c.customer_id
FROM customers c
LEFT JOIN transactions t
ON c.customer_id = t.customer_id
WHERE t.customer_id IS NULL;
```

Output

customer_id
5

Q21

```
SELECT department_id,
       AVG(salary) AS avg_salary
FROM employees
GROUP BY department_id
ORDER BY avg_salary DESC
LIMIT 1;
```

Output

department_id	avg_salary
1	84250

Q22

```
SELECT department_id,
       MIN(salary)
FROM employees
GROUP BY department_id
HAVING MIN(salary) > 40000;
```

Output

department_id	MIN(salary)
NULL	50000
1	72000
2	45000
3	62000

Q23

```
SELECT *
FROM employees
WHERE joining_date >= '2026-01-01'
      AND joining_date < '2027-01-01';
```

Output

employee_id	employee_name	department_id	salary	joining_date	city
101	Amit	1	85000	2026-03-15	Delhi
103	Priya	2	65000	2026-01-20	Pune
105	Amit	1	88000	2026-03-15	Mumbai
106	Ravi	NULL	50000	2026-07-01	Pune
108	Vijay	2	45000	2026-05-18	Delhi
110	Pooja	3	62000	2026-04-12	Mumbai

Q24

```
SELECT *  
FROM employees  
ORDER BY joining_date DESC  
LIMIT 1;
```

Output

employee_id	employee_name	department_id	salary	joining_date	city
106	Ravi	NULL	50000	2026-07-01	Pune

Q25

```
SELECT employee_id  
FROM projects  
GROUP BY employee_id  
HAVING COUNT(DISTINCT project_id) >= 3;
```

Output

employee_id
101

Q26

```
SELECT e.employee_id  
FROM employees e  
LEFT JOIN projects p  
ON e.employee_id = p.employee_id  
WHERE p.project_id IS NULL;
```

Output

employee_id
105
106
107
108
109
110

Q27

```
SELECT *  
FROM projects  
ORDER BY project_cost DESC  
LIMIT 1;
```

Output

project_id	project_name	employee_id	project_cost	start_date
5	Analytics	103	1200000	2026-02-01

Q28

```
SELECT d.department_id,  
       d.department_name,  
       COUNT(e.employee_id) AS employee_count  
FROM departments d  
LEFT JOIN employees e  
ON d.department_id = e.department_id  
GROUP BY d.department_id, d.department_name;
```

Output

department_id	department_name	employee_count
1	Engineering	4
2	HR	2
3	Finance	3
4	Marketing	0

Q29

```
SELECT department_id,  
       MAX(salary) - MIN(salary) AS salary_range  
FROM employees  
GROUP BY department_id;
```

Output

department_id	salary_range
NULL	0
1	20000
2	20000
3	33000

Q30

```
SELECT department_id  
FROM employees  
GROUP BY department_id  
HAVING COUNT(*) >= 5  
       AND AVG(salary) > 60000;
```

Output

(Empty result — 0 rows)

Q31

```
SELECT employee_id,  
       employee_name,  
       department_id,  
       salary,  
       RANK() OVER (  
         PARTITION BY department_id  
         ORDER BY salary DESC  
       ) AS salary_rank  
FROM employees;
```

Output

employee_id	employee_name	department_id	salary	salary_rank
106	Ravi	NULL	50000	1
107	Anita	1	92000	1
105	Amit	1	88000	2
101	Amit	1	85000	3
102	Rahul	1	72000	4
103	Priya	2	65000	1
108	Vijay	2	45000	2
104	Neha	3	95000	1
109	Kiran	3	78000	2
110	Pooja	3	62000	3

Q32

```
SELECT *
FROM (
    SELECT e.*,
           DENSE_RANK() OVER (
               PARTITION BY department_id
               ORDER BY salary DESC
           ) AS salary_rank
    FROM employees e
) x
WHERE salary_rank <= 3;
```

Output

employee_id	employee_name	department_id	salary	joining_date	city	salary_rank
106	Ravi	NULL	50000	2026-07-01	Pune	1
107	Anita	1	92000	2023-02-14	Bangalore	1
105	Amit	1	88000	2026-03-15	Mumbai	2
101	Amit	1	85000	2026-03-15	Delhi	3
103	Priya	2	65000	2026-01-20	Pune	1
108	Vijay	2	45000	2026-05-18	Delhi	2
104	Neha	3	95000	2024-11-05	Delhi	1
109	Kiran	3	78000	2025-09-20	Bangalore	2
110	Pooja	3	62000	2026-04-12	Mumbai	3

Q33

```
SELECT employee_id,
       salary,
       SUM(salary) OVER (
           ORDER BY employee_id
       ) AS running_total
FROM employees;
```

Output

employee_id	salary	running_total
101	85000	85000
102	72000	157000
103	65000	222000
104	95000	317000
105	88000	405000
106	50000	455000
107	92000	547000
108	45000	592000
109	78000	670000
110	62000	732000

Q34

```
SELECT employee_id,  
       salary,  
       AVG(salary) OVER (  
         PARTITION BY department_id  
       ) AS department_avg  
FROM employees;
```

Output

employee_id	salary	department_avg
106	50000	50000
101	85000	84250
102	72000	84250
105	88000	84250
107	92000	84250
103	65000	55000
108	45000	55000
104	95000	78333.33
109	78000	78333.33
110	62000	78333.33

Q35

```
SELECT *  
FROM employees e  
WHERE salary > (  
  SELECT AVG(salary)  
  FROM employees  
  WHERE department_id = e.department_id  
);
```

Output

employee_id	employee_name	department_id	salary	joining_date	city
101	Amit	1	85000	2026-03-15	Delhi
103	Priya	2	65000	2026-01-20	Pune
104	Neha	3	95000	2024-11-05	Delhi
105	Amit	1	88000	2026-03-15	Mumbai
107	Anita	1	92000	2023-02-14	Bangalore

Q36

```
SELECT *  
FROM (  
  SELECT h.*,  
         ROW_NUMBER() OVER (  
           PARTITION BY employee_id  
           ORDER BY effective_date DESC  
         ) AS rn  
  FROM employee_history h  
) x  
WHERE rn = 1;
```

Output

employee_id	salary	effective_date	rn
101	85000	2026-03-15	1
102	72000	2025-06-10	1
103	65000	2026-01-20	1

Q37

```
WITH high_salary AS (  
    SELECT *  
    FROM employees  
    WHERE salary > 80000  
)  
SELECT *  
FROM high_salary;
```

Output

employee_id	employee_name	department_id	salary	joining_date	city
101	Amit	1	85000	2026-03-15	Delhi
104	Neha	3	95000	2024-11-05	Delhi
105	Amit	1	88000	2026-03-15	Mumbai
107	Anita	1	92000	2023-02-14	Bangalore

Q38

```
SELECT department_id,  
       AVG(salary) AS avg_salary  
FROM employees  
GROUP BY department_id  
ORDER BY avg_salary DESC  
LIMIT 1;
```

Output

department_id	avg_salary
1	84250

Q39

```
SELECT employee_id, joining_date  
FROM employees  
GROUP BY employee_id, joining_date  
HAVING COUNT(*) > 1;
```

Output

(Empty result — 0 rows)

Q40

```
SELECT employee_id,  
       employee_name,  
       joining_date,  
       ROW_NUMBER() OVER (  
           PARTITION BY employee_id  
           ORDER BY joining_date  
       ) AS row_num  
FROM employees;
```

Output

employee_id	employee_name	joining_date	row_num
101	Amit	2026-03-15	1
102	Rahul	2025-06-10	1
103	Priya	2026-01-20	1
104	Neha	2024-11-05	1
105	Amit	2026-03-15	1
106	Ravi	2026-07-01	1
107	Anita	2023-02-14	1
108	Vijay	2026-05-18	1
109	Kiran	2025-09-20	1
110	Pooja	2026-04-12	1

Q41

```
SELECT customer_id,  
       MAX(amount) AS highest_transaction  
FROM transactions  
GROUP BY customer_id;
```

Output

customer_id	highest_transaction
1	160000
2	250000
3	150000
4	30000

Q42

```
SELECT *  
FROM (  
    SELECT t.*,  
           ROW_NUMBER() OVER (  
               PARTITION BY customer_id  
               ORDER BY amount DESC  
           ) AS rn  
    FROM transactions t  
) x  
WHERE rn = 1;
```

Output

transaction_id	customer_id	transaction_date	transaction_type	amount	rn
1003	1	2026-02-10	CREDIT	160000	1
1004	2	2026-01-12	CREDIT	250000	1
1006	3	2026-02-05	CREDIT	150000	1
1007	4	2026-03-01	CREDIT	30000	1

Q43

```
SELECT YEAR(transaction_date) AS yr,  
       MONTH(transaction_date) AS mon,  
       SUM(amount) AS total_amount  
FROM transactions  
GROUP BY YEAR(transaction_date),  
         MONTH(transaction_date);
```

Output

yr	mon	total_amount
2026	1	395000
2026	2	310000
2026	3	30000

Q44

```
SELECT customer_id,  
       YEAR(transaction_date) AS yr,  
       MONTH(transaction_date) AS mon,  
       SUM(amount) AS total_amount,  
       RANK() OVER (  
         PARTITION BY YEAR(transaction_date),  
                       MONTH(transaction_date)  
         ORDER BY SUM(amount) DESC  
       ) AS customer_rank  
FROM transactions  
GROUP BY customer_id,  
       YEAR(transaction_date),  
       MONTH(transaction_date);
```

Output

customer_id	yr	mon	total_amount	customer_rank
2	2026	1	250000	1
3	2026	1	75000	2
1	2026	1	70000	3
1	2026	2	160000	1
3	2026	2	150000	2
4	2026	3	30000	1

Q45

```
SELECT customer_id,  
       transaction_date,  
       amount,  
       SUM(amount) OVER (  
         PARTITION BY customer_id  
         ORDER BY transaction_date  
       ) AS running_total  
FROM transactions;
```

Output

customer_id	transaction_date	amount	running_total
1	2026-01-10	50000	50000
1	2026-01-15	20000	70000
1	2026-02-10	160000	230000
2	2026-01-12	250000	250000
3	2026-01-20	75000	75000
3	2026-02-05	150000	225000
4	2026-03-01	30000	30000

Q46

```
SELECT project_id,  
       start_date,  
       project_cost,  
       SUM(project_cost) OVER (  
         ORDER BY start_date  
       ) AS cumulative_cost  
FROM projects;
```

Output

project_id	start_date	project_cost	cumulative_cost
1	2026-01-10	400000	400000
4	2026-01-20	300000	700000
5	2026-02-01	1200000	1900000
2	2026-02-15	700000	2600000
3	2026-03-01	150000	2750000
6	2026-03-10	50000	2800000

Q47

```
SELECT customer_id,  
       transaction_date,  
       amount,  
       LAG(amount) OVER (  
         PARTITION BY customer_id  
         ORDER BY transaction_date  
       ) AS previous_amount  
FROM transactions;
```

Output

customer_id	transaction_date	amount	previous_amount
1	2026-01-10	50000	NULL
1	2026-01-15	20000	50000
1	2026-02-10	160000	20000
2	2026-01-12	250000	NULL
3	2026-01-20	75000	NULL
3	2026-02-05	150000	75000
4	2026-03-01	30000	NULL

Q48

```
SELECT customer_id,  
       transaction_date,  
       amount,  
       LEAD(amount) OVER (  
         PARTITION BY customer_id  
         ORDER BY transaction_date  
       ) AS next_amount  
FROM transactions;
```

Output

customer_id	transaction_date	amount	next_amount
1	2026-01-10	50000	20000
1	2026-01-15	20000	160000
1	2026-02-10	160000	NULL
2	2026-01-12	250000	NULL
3	2026-01-20	75000	150000
3	2026-02-05	150000	NULL
4	2026-03-01	30000	NULL

Q49

```
SELECT *
FROM transactions
WHERE amount IS NULL
      OR customer_id IS NULL
      OR amount < 0;
```

Output

(Empty result — 0 rows)

Q50

```
SELECT *
FROM (
    SELECT e.*,
           DENSE_RANK() OVER (
               PARTITION BY department_id
               ORDER BY salary DESC
           ) AS rnk
    FROM employees e
) x
WHERE rnk <= 2;
```

Output

employee_id	employee_name	department_id	salary	joining_date	city	rnk
106	Ravi	NULL	50000	2026-07-01	Pune	1
107	Anita	1	92000	2023-02-14	Bangalore	1
105	Amit	1	88000	2026-03-15	Mumbai	2
103	Priya	2	65000	2026-01-20	Pune	1
108	Vijay	2	45000	2026-05-18	Delhi	2
104	Neha	3	95000	2024-11-05	Delhi	1
109	Kiran	3	78000	2025-09-20	Bangalore	2